Agent Control 3.1.0 Operator Guide

Infrastructure-neutral installation, dashboard, scheduling, operation and recovery

Release 3.1.0

24 August 2026

Execution may be delegated. Authority is not.

Purpose

Agent Control coordinates durable agent and repeatable Job work while retaining human and policy authority. This guide covers installation, deployment patterns, configuration, the TUI, the web dashboard, Jobs and Schedules, monitoring, recovery, evidence and safe shutdown.

Authority boundary

Agent Control decides lanes, priorities, scheduling, leases, PTY ownership, human takeover, placement, approvals, verification, recovery validity and experiment winners. A browser, model, worker or execution substrate may request or execute work but cannot grant itself authority.

The dashboard requests; Agent Control authorises. Orca may execute; Agent Control decides.

Requirements

- Node.js 20 or newer;
- npm and Git;
- Bash for shell validation and optional Android helpers;
- optional SSH, Orca, Android/Termux or model/provider services only when configured.

No host, device, provider, port, GPU, repository path or network overlay is built in.

Install

```
git clone https://github.com/lozknowles/agent-control.git
cd agent-control
npm install
mkdir -p .agent-control
cp config/agent-control.example.json .agent-control/config.json
npm run check
```

Windows PowerShell:

```
git clone https://github.com/lozknowles/agent-control.git
Set-Location agent-control
npm install
New-Item -ItemType Directory -Force .agent-control
Copy-Item config/agent-control.example.json .agent-control/config.json
npm run check
```

Never put credentials in repository configuration. Use the named environment variable or an approved operating-system secret manager.

Configure

The default configuration is `.agent-control/config.json`. Override the configuration path with `AGENT_CONTROL_CONFIG` and the durable runtime directory with `AGENT_CONTROL_STATE_DIR`.

Resources have logical IDs, independent transports and semantic capabilities. Providers describe qualified API/CLI capabilities. Services contain explicit health URLs and optional reviewed start recipes. Lanes contain identity, working directory, priority and AUTO/MANUAL mode. Empty collections are valid and fail safely to `UNCONFIGURED`.

Job and Schedule manifests live under `config/jobs/` by default. They are repository-reviewed YAML or JSON validated against the versioned schemas under `config/schemas/`. Set `AGENT_CONTROL_JOB_DIR` only when an alternative reviewed catalog is required.

Start and deploy

```
npm run status
npm run up
npm start
```

`status` is read-only. `up` starts only explicit configured recipes and records ownership. `npm start` opens the TUI and starts the same localhost dashboard service. For a headless operator host use:

```
npm run web
```

Run one authoritative Agent Control process per state directory. Keep `.agent-control/` on durable operator-only storage. A source release is not a production deployment.

Supported patterns include a local workstation, a controller with capability-advertising workers, API-only providers and an optional Android resource. Use a persistent terminal multiplexer for the interactive TUI when controller disconnect/reconnect is required.

Web dashboard

The dashboard binds to `127.0.0.1:4310` by default. It is observer-only unless an operator token is explicitly configured:

```
export AGENT_CONTROL_WEB_OPERATOR_TOKEN="$(openssl rand -hex 32)"
npm start
```

Enter the token through **Observer mode**. It remains in browser-tab session storage and is sent only as a bearer header. Agent Control creates no authority cookie. Use `AGENT_CONTROL_WEB_ENABLED=0` to disable the dashboard or `AGENT_CONTROL_WEB_PORT` to select a different port.

Do not expose the listener beyond localhost without an explicitly reviewed authenticated TLS boundary and `AGENT_CONTROL_WEB_ALLOWED_ORIGINS`. Observer reads do not grant operator authority.

The **Jobs** view is the default and shows:

- repository-managed Jobs and enabled state;
- schedule expression, timezone, previous and next occurrence;
- authoritative Run and step state;
- worker placement, attempts, duration and verification;
- queue priority, age, wait reason, eligible workers and missing capabilities;
- worker health/capacity and semantic resource locks;
- searchable Run history;
- artifact type, schema, size, checksum, retention and provenance.

Run now, schedule enable/disable, cancellation, whole-Run retry and exact named approval call authenticated `AgentControlService` methods. The browser does not own scheduler state and has no PTY-input, lease or ownership endpoint.

The **Lanes** view shows the same lane, provider, baton, routing, Git, PTY ownership, context and verification projection as the TUI. Its terminal panel is observer-only. Human takeover invokes the authoritative PTY fence and remains unconditional.

TUI controls

Key	Action
Tab	select next lane
I / Enter	open command input
T	inspect PTYs
J	inspect Jobs, Schedules and Runs
W	inspect Work Queue detail
G	probe configured providers
Y	run configured Responses proof
D	inject isolated demo workload
X	probe configured Android resource
Z	request allow-listed Android recovery
A	toggle Android recovery AUTO/MANUAL
R	request capability/provider substitution
P	pause/resume and checkpoint
Q / Ctrl-C	persist and quit

The TUI and web UI are clients of one control service. Neither contains scheduling, lease, ownership or verification authority.

Adaptive harness execution

For normal Work Queue agent work, `WorkExecutor` first uses scheduler-selected worker placement and asks `AdaptiveHarness` to build a fingerprinted `ExecutionRecipe`. The recipe records provider/model routing, prompt profile, selected context, qualified skills, granted tools, safe runtime settings, authority generations and verification/escalation requirements. Recipe construction does not claim work, alter placement, acquire a lease, write a PTY or accept a result.

The executor receives only a `ToolInvocationGateway`. Each invocation is checked against the recipe grant and current capability, privilege, worker, lease, ownership and human-takeover state. Policy denial fails closed and never falls back to a raw handler. Non-agent maintenance such as Android provisioning uses an explicitly named and scope-checked control operation.

Recipe execution ends in `verification-pending`; use the evidence/verification workflow before acceptance. The recipe-dispatch store is an inspectable bridge to the Run Ledger and must remain on operator-only durable storage. It contains identities and policy metadata, never API keys or credentials.

Windows OpenAI return-data example

Agent Control can run on Windows and select between two official OpenAI execution paths:

- `api-key`: the OpenAI Responses API, billed against API quota;

- `chatgpt-plan`: official `codex exec`, reusing the Codex client's saved ChatGPT sign-in and included plan allowance.

The default `OPENAI_AUTH_MODE=auto` chooses the API path when a non-empty `OPENAI_API_KEY` is available and otherwise chooses the ChatGPT-plan path. Set `OPENAI_AUTH_MODE=api-key` or `OPENAI_AUTH_MODE=chatgpt-plan` to force a route during qualification. Forced API mode fails closed when the key is absent. This is not automation of the ChatGPT desktop window: Agent Control never scrapes the UI or reuses browser cookies/session tokens.

Add a provider to the operator-owned configuration. The credential stays outside this file:

```
{
  "id": "openai-responses",
  "name": "OpenAI Responses API",
  "kind": "responses",
  "baseUrl": "https://api.openai.com/v1",
  "wireApi": "responses",
  "requiresAuth": true,
  "costClass": "metered",
  "capabilities": ["structured-output", "tool-request"]
}
```

For automatic selection, make an official Codex CLI executable available on `PATH` (or set `CODEX_COMMAND`) and authenticate it once with ChatGPT:

```
codex login
codex login status
$env:OPENAI_AUTH_MODE = "auto"
npm run qualify:openai-windows
Remove-Item Env:OPENAI_AUTH_MODE
```

The package command reads an ignored `.env.local` when present. If it contains `OPENAI_API_KEY`, `auto` uses the Responses API. Without that key it runs `codex exec --ephemeral --json --sandbox read-only --ignore-user-config`, confirms that the saved login is ChatGPT-managed, and requests schema-constrained return data. API-key variables are removed from the Codex child environment so the fallback cannot silently become API-billed. Optional model selectors are `OPENAI_API_MODEL` and `CODEX_CHATGPT_MODEL`.

To bypass a present but quota-depleted API key and deliberately use the ChatGPT plan:

```
$env:OPENAI_AUTH_MODE = "chatgpt-plan"
$env:CODEX_CHATGPT_MODEL = "gpt-5.6-terra"
npm run qualify:openai-windows
Remove-Item Env:OPENAI_AUTH_MODE, Env:CODEX_CHATGPT_MODEL
```

The qualification follows this path:

```
Job -> HarnessJobAgentAction -> HarnessDispatcher -> AdaptiveHarness
    -> ExecutionRecipe -> Responses API function_call
                       or Codex schema-constrained return request
    -> ToolInvocationGateway -> ToolPolicy -> tool handler
    -> typed checksummed artifact -> verification
```

The model receives a bounded request contract, not raw Agent Control handlers. Its function call or structured return is only a request. Agent Control rechecks the recipe grant, live worker, lease generation, ownership generation, risk approval and human ownership before invoking the handler. The Action declares the output schema; `ArtifactStore` persists the returned JSON with SHA-256 and provenance. Execution remains distinct from verification and acceptance.

Both switchable routes are `SUPPORTED+QUALIFIED`. A real Responses API run with the existing ignored key and `gpt-4o-mini` returned a function call through `ToolInvocationGateway`, produced a checksummed artifact and reached verified Run success. A separate real Windows `auto` run with no API key selected official Codex/ChatGPT authentication and passed the same authority and verification boundaries. See

`docs/evidence/windows-openai-harness-qualification-20260824.md`.

Codex is an opaque CLI execution family. Its built-in operations are constrained by an ephemeral read-only process envelope and user-configured MCP tools are disabled for this path; they are not falsely claimed as individually mediated by Agent Control. Any returned Agent Control tool request still passes through the central live `ToolPolicy` gate, so stale authority or human ownership prevents the requested handler from running.

An operator-supplied local Windows bridge may instead be configured as `kind: "browser-bridge"`, `wireApi: "responses"` on loopback. That bridge must be independently approved and qualified. Cleartext non-loopback bridge endpoints are rejected. Agent Control does not ship a ChatGPT desktop-window bridge; the qualified subscription path is official Codex non-interactive execution, and desktop UI automation remains `NOT TESTED`.

Jobs, Actions and Runs

An Action is a versioned executable capability. A Job is a declarative DAG of Actions. A separate Schedule decides when to create a Run. Manual and scheduled triggers call the same `createRun` path.

Current registered Actions are control-owned handlers. A new Action that invokes an agent or model must delegate through `HarnessDispatcher`; Action registration does not qualify a model, grant tools or confer authority. Tools used opaquely inside an external CLI are not yet claimed as individually moderated.

Jobs request capabilities and semantic resources, never machine names. Credentials are represented as approved capability bindings or secret references and are never embedded in manifests. Historical Runs retain the effective Job version, parameters, trigger, selected workers, attempts, artifacts, verification and provenance.

Run states distinguish queued/running/verifying/succeeded/failed/degraded/cancelled/missed/disconnected. Steps additionally expose dependency, worker, resource, approval and retry waits. The dashboard consumes these structures; it does not parse terminal text to infer progress.

Scheduler operation

The Agent Control scheduler owns due Schedule discovery, Run creation, dependency evaluation, priority ordering, capability placement, concurrency, resource locks, approvals, retries and outcome recording. OS cron and the browser are not authoritative schedulers.

Schedules are timezone-aware and retain previous/next time, last Run, last success/failure and missed count. Missed policies are `skip`, `run-next-available` and `run-once-immediately`. The scheduler never blindly replays every historical occurrence.

Concurrency policies are `allow`, `no-overlap`, `replace-running` and `queue`. Production-like workflows should normally use `no-overlap` or `queue`. Replacement requests cancellation but retains exclusive locks until live execution confirms return.

A waiting Run explains whether it lacks dependencies, a healthy eligible worker, semantic resource, approval or retry time. Model/provider routing remains separate from worker placement.

Artifacts and verification

Steps exchange explicit typed artifacts rather than assumed shared paths. Artifact metadata records Run, step, type/schema/version, size, SHA-256, retention, managed storage and provenance. Every read revalidates the checksum. Browser projections hide managed storage paths.

Process exit is not success. A declared verification observation must be present before a step succeeds. Lane results separately move through claimed, evidence collected, verified and accepted. Git/tests remain stronger evidence than agent interpretation or optional shared-thread context.

Monitor

```
npm run status
npm run qualify
npm run qualify:jobs
```

Monitor lane status, baton health, queue age, Job/Run state, outstanding approvals, worker health/capacity, locks, artifact provenance, provider qualification, PTY owner, execution recovery and verification phase. Qualification output is written beneath ignored `qualification-results/`; review it for credentials and private topology before preserving selected evidence.

Healthy transport is not proof of original execution identity. After restart/reconnect, autonomous continuation requires the persisted task/session/resource/repository/worktree/branch/nonce plus current lease and ownership generations as applicable. PID alone is insufficient.

Human takeover and recovery

Human takeover increments the ownership generation, fences agent writers and remains in force across adapter reconnect/restart. Ownership returns only through a deliberate Agent Control operation followed by scheduler revalidation.

An unproven in-flight Job execution becomes `DISCONNECTED`; its execution identity is not assumed and durable resource locks remain for reconciliation. Resolve unknown state manually or through a validated execution-provider reconnect.

Stop and rollback

```
npm run down
```

`down` stops only processes recorded as Agent-Control-owned. It does not stop unrelated listeners. Before upgrade, pause/checkpoint work, preserve the state directory, verify the target tag/SHA and run the complete gate. Roll back source only to a verified compatible revision and never claim execution survival without identity validation.

Optional execution and Android

The execution-provider contract is start, status, reconnect, input, pause, resume, cancel, output, diff and cleanup. Orca-specific concepts remain behind its adapter; the built-in executor stays available as fallback.

Android is an optional device-neutral resource. Provisioning uses explicit privilege, pairing and reboot approvals:

```
npm run provision:android
```

The bundled node is loopback-bound by default and exposes only its allow-listed read-only operation.

Troubleshooting

- `UNCONFIGURED`: create or select a valid configuration.
- provider unavailable: check configured health, then run a redacted functional proof.

- Job waiting for worker: inspect required, missing and expired capabilities.
- Job waiting for resource: inspect the lock holder before cancelling or reconciling it.
- approval rejected: approve only the exact policy of a step already waiting for approval.
- execution `DISCONNECTED` or `UNKNOWN`: do not resume automatically; validate identity evidence.
- dashboard mutation denied: configure the token and allowed origin; do not weaken authentication.
- context unavailable: continue from baton, Git and tests and record the accessibility failure.
- harness policy denied: inspect missing qualification, capability, tool/risk approval and live authority; do not substitute an unrestricted legacy handler.
- recipe is stale after takeover or lease change: build a new policy-authorised recipe after ownership is deliberately returned and revalidated.

Release validation

```
npm install
npm run typecheck
npm run check:bootstrap
npm run check:dashboard
npm run check:neutrality
npm test
npm run check
npm run qualify:jobs
npm run qualify:openai-windows
git diff --check
```

Review the architecture boundary, qualification evidence, changed-file list, ignored runtime files and secret scan before committing or tagging. The safe events qualification uses fixtures, performs no network publication and ships with its Schedule disabled.